

SliceMatic Ordering System - Product Requirements and Business Economics

Client: SliceMatic

Document purpose: Business requirements, ordering workflow, and unit economics review

Version: Client review draft

Executive Snapshot

SliceMatic needs a digital ordering system that replaces phone-only ordering with a reliable customer intake, menu selection, pricing, GST, payment confirmation, and order logging workflow. The initial product scope is intentionally focused: it must be precise enough for MVP development while also being realistic enough to support a future production release with database-backed analytics and admin controls.

The companion economics document is a good baseline, but it has internal inconsistencies that should be handled explicitly:

- SliceMatic prices a pizza as base plus pizza plus topping. Using the provided menu catalogue, the average single-pizza basket is Rs.568.50 before GST, not Rs.420.
- The reference model's headline AOV of about Rs.847 is directionally plausible. The operating model below uses the weekday/weekend weighted AOV of Rs.850.34, equal to about 1.50 pizzas per order.
- GST should be added at billing and excluded from P&L revenue. Therefore, menu prices and AOV are treated as ex-GST in this analysis.
- The reference contribution margin of Rs.644/order appears optimistic because it does not fully scale ingredient and packaging cost with multi-pizza orders. A more conservative refined contribution margin is Rs.589/order.
- Refined break-even is about 12 orders/day, still healthy against the 60% capacity plan of roughly 46 orders/day.

Product Requirements

Product Vision

SliceMatic Ordering System is a guided digital pizza-ordering experience for SliceMatic customers. It replaces phone-based ordering with a structured workflow that captures customer details, presents the active menu, validates quantity and selections, calculates discounts and GST accurately, confirms payment mode, and saves every completed order in a parseable format for future business analysis. The system should reduce order-taking errors, make billing transparent, and create the data foundation for future analytics.

Business Objectives and Success Metrics

Objective	Why It Matters	Success Metric
Reduce manual order-taking errors	Phone orders create pricing, selection, and customer-detail mistakes.	100% of completed orders have validated customer data, quantity, selections, bill total, and payment mode.
Improve billing transparency	Customers should understand exactly what they are paying for.	Every completed order shows subtotal, discount, GST, and final payable before confirmation.
Protect margin discipline	Discounts and GST should never be applied manually or inconsistently.	Discount and GST are calculated by system rules, not staff judgement.
Build useful order data	Management needs a reliable base for menu, staffing, and demand decisions.	Each completed order is saved in a parseable format with timestamp, item details, amount, and payment mode.
Prepare for production scaling	The MVP should prove the workflow before investment in database, admin, and AI features.	MVP logic can migrate to a database-backed production release without changing core pricing rules.

Primary Users and Stakeholders

User / Stakeholder	Need	Success Outcome
Customer	Place a pizza order without confusion	Clear menu, clear bill, quick confirmation
Counter staff	Avoid manual pricing and phone mistakes	Validated inputs and automatic bill generation
Owner / manager	Understand revenue, demand, and menu mix	Complete order log for analysis
Kitchen team	Receive unambiguous order details	Item, quantity, and payment status are clear
Delivery riders	Receive deliverable orders	Customer phone is captured; address capture and radius validation are flagged for production

Scope

In scope for the MVP:

- Customer name and phone intake.
- Quantity selection from 1 to 10 pizzas.
- Runtime loading of the active menu catalogue from controlled source files.
- Menu browsing and selection by item number.
- Itemized billing with subtotal, discount, GST, and final payable.
- Payment mode selection among Cash, Card, and UPI.
- Completed order persistence to `orders\_log.txt`.
- Defensive validation and graceful failure for invalid inputs and malformed files.

Out of scope for the MVP but required or relevant for the production release:

- Real payment gateway integration.
- Inventory tracking.
- Rider dispatch and delivery status tracking.
- Address capture and delivery-zone validation; the MVP assumes staff confirm delivery details manually after order confirmation.
- Customer login, loyalty, coupons, refunds, and cancellations.
- Admin authentication and database-backed analytics.
- Production observability, audit logs, and privacy controls.

Functional Requirements

ID	Requirement	Detailed Rule	Priority
FR-01	Start session	On customer entry, create a session timestamp in India time and begin the guided ordering journey.	Must
FR-02	Load active menu catalogue	The MVP must read `Types_of_Base.txt`, `Types_of_Pizza.txt`, and `Types_of_Toppings.txt` at runtime using `ID;Name;Price` format.	Must
FR-03	Validate menu catalogue	Each menu row must have exactly three fields after trimming whitespace. Price must be numeric and greater than or equal to zero.	Must
FR-04	Handle unavailable menu data	If menu data is missing or malformed, show a controlled service message and prevent order completion.	Must
FR-05	Capture customer name	Accept alphabets and spaces only, minimum 2 and maximum 40 characters after trimming.	Must
FR-06	Capture phone	Accept exactly 10 digits starting with 6, 7, 8, or 9.	Must

ID	Requirement	Detailed Rule	Priority
FR-07	Select quantity	Accept integer quantity from 1 to 10 only. Reject empty input, text, floats, zero, negatives, and values above 10.	Must
FR-08	Display menu options	Show bases, pizzas, and toppings as numbered lists with item name and INR price.	Must
FR-09	Select menu items	Accept item number only. Reject letters, empty input, price values, zero, and numbers greater than menu length.	Must
FR-10	Calculate unit price	Unit price equals selected base price plus selected pizza price plus selected topping price.	Must
FR-11	Calculate subtotal	Subtotal equals unit price multiplied by quantity.	Must
FR-12	Apply discount	If quantity is greater than or equal to 5, apply a 10% discount on subtotal. Otherwise discount is Rs.0.	Must
FR-13	Calculate GST	GST equals 18% of post-discount subtotal. GST is calculated after discount, not before discount.	Must
FR-14	Calculate final total	Final payable equals post-discount subtotal plus GST. Round display values to two decimals.	Must
FR-15	Present itemized bill	Show base, pizza, topping, unit prices, quantity, subtotal, GST, and final payable in a clean bill table. Show the discount line only when quantity is greater than or equal to 5.	Must
FR-16	Payment mode	Offer exactly three payment choices: Cash, Card, UPI.	Must
FR-17	Payment validation	Reject invalid payment selections and ask the user to retry.	Must
FR-18	Payment confirmation	Show a mode-specific confirmation message: Cash means "Cash payment selected; collect at delivery/counter", Card means "Card payment selected; process on POS", and UPI means "UPI payment selected; confirm receipt before fulfillment."	Must
FR-19	Persist completed order	Append every successfully completed order to `orders_log.txt`.	Must
FR-20	Keep log parseable	Use one pipe-separated line per completed order, followed by one blank line between order records.	Must
FR-21	Do not log failed attempts as completed orders	Validation failures should not create completed order records.	Must

Pricing Logic

All menu prices are treated as ex-GST. GST is charged at checkout.

```

unit_price = base_price + pizza_price + topping_price
subtotal = unit_price * quantity
discount = subtotal * 0.10 when quantity >= 5, otherwise 0
taxable_amount = subtotal - discount
gst = taxable_amount * 0.18
final_payable = taxable_amount + gst

```

Sample using the reference bill:

Component	Amount
Cheese Burst base	Rs.229.00

Component	Amount
BBQ Chicken pizza	Rs.379.00
Extra Cheese topping	Rs.69.00
Unit price	Rs.677.00
Quantity	5
Subtotal	Rs.3,385.00
Discount at 10%	Rs.338.50
Post-discount subtotal	Rs.3,046.50
GST at 18%	Rs.548.37
Final payable	Rs.3,594.87

Order Log Format

Recommended completed-order format. In the actual log, store this as one pipe-separated line per completed order, followed by one blank line; it is wrapped below only for readability:

```
ORDER|timestamp|customer_name|phone|
base_id|base_name|base_price|
pizza_id|pizza_name|pizza_price|
topping_id|topping_name|topping_price|
quantity|unit_price|subtotal|discount|gst|final_total|payment_mode
```

Example:

```
ORDER|2026-06-25T19:42:10+05:30|Aman Sharma|9876543210|
B3|Cheese Burst|229.00|P7|BBQ Chicken|379.00|
T2|Extra Cheese|69.00|5|677.00|3385.00|338.50|548.37|3594.87|UPI
```

Non-Functional Requirements

Area	Requirement
Reliability	No unhandled exception should be visible to the user during normal or invalid inputs.
Validation	Every input field must have a specific validation message, not a generic crash or blank failure.
File parsing	Strip whitespace, reject missing fields, reject non-numeric prices, and avoid hardcoded menu content.
Calculation accuracy	Use numeric types consistently and round only for display/logging, not midway through formulas.
Usability	The order flow should be step-based, not a single dense form. The bill should be displayed in a structured table, not as unformatted text.
Graceful failure	If startup data is invalid, show the file and row problem and prevent order completion.
Data quality	Completed order records must include all required bill fields so they can be parsed for analytics.
Privacy	Phone numbers are personally identifiable information and should not be exposed outside the order workflow/log.
Performance	The MVP should comfortably handle small local usage. The production release must move persistence to a real database.
Maintainability	Business constants such as GST rate, discount rate, and max quantity should be named variables.

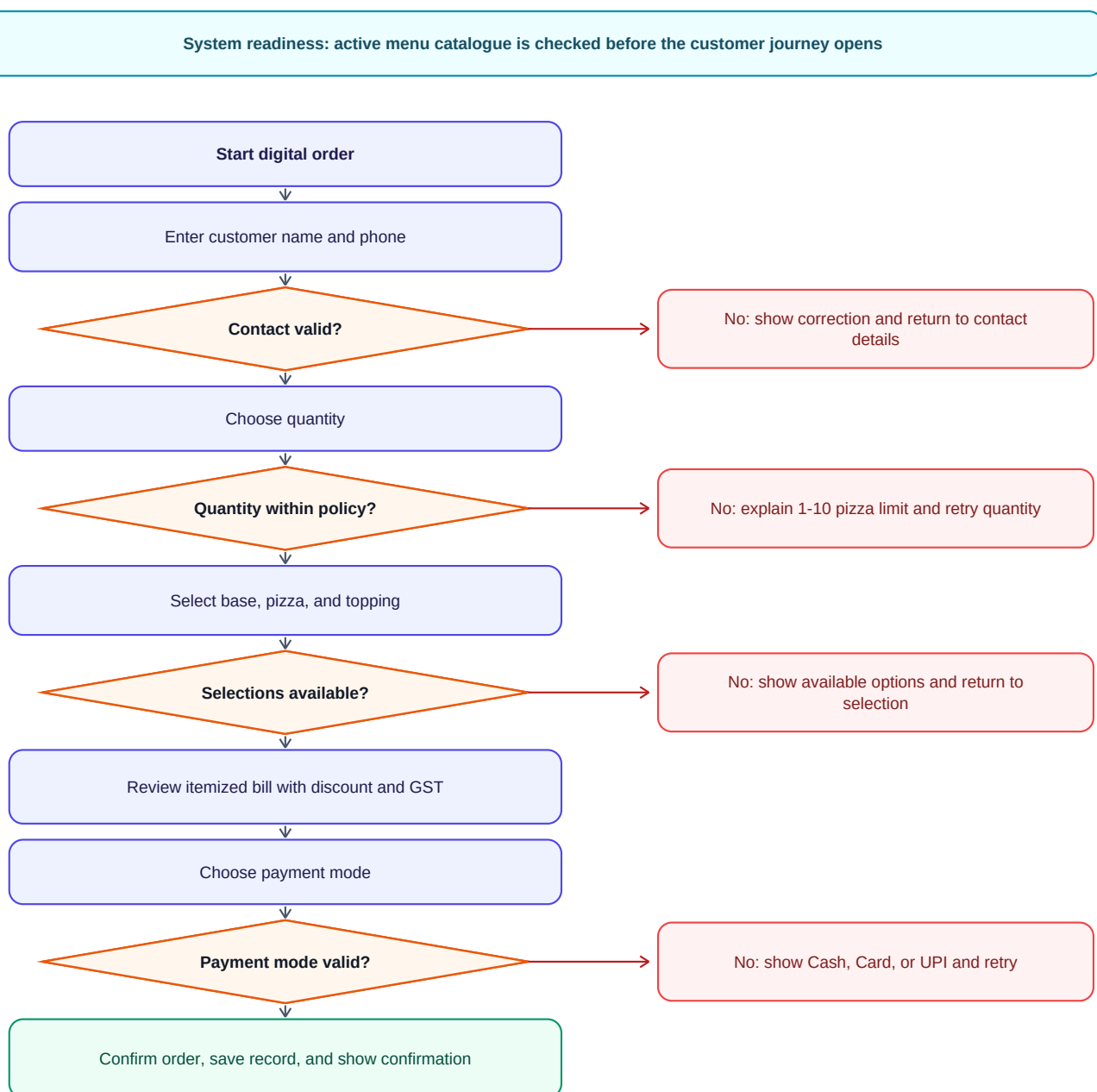
Edge Cases to Handle

Edge Case	Expected Behavior
Name contains only spaces	Reject with "Name must contain alphabetic characters and be 2-40 characters long."
Phone has 10 digits but starts with 1	Reject with "Enter a valid Indian mobile number starting with 6, 7, 8, or 9."

Edge Case	Expected Behavior
Quantity is 0	Reject with "Quantity must be between 1 and 10."
Quantity is 11	Reject with "Maximum outlet capacity is 10 pizzas per order."
Quantity is "three" or "2.5"	Reject with "Quantity must be a whole number from 1 to 10."
Item selection is 0	Reject with "Select a valid item number from the list."
Item selection is greater than menu length	Reject with "That item number is not available."
User enters a price instead of item number	Reject unless that value is also a valid displayed item number.
Empty input at any prompt	Reject with a field-specific message.
Menu row misses price	Stop startup gracefully and show the faulty file/line.

Customer Ordering Flow

The client-facing flow should describe the customer's journey, not the application's internal startup logic. Menu-source validation is a system readiness control that happens before the ordering journey is opened to the customer.



Yes path continues downward. Invalid entries return to the same business step; failed attempts are not saved as completed orders.

System Controls Behind the Flow

Control	Why the Client Needs It	Implementation Requirement
Menu catalogue readiness	Customers should never reach a broken menu or see developer-style file errors.	Before accepting an order, verify that the active base, pizza, and topping catalogue is available and valid. If not, show a controlled service-unavailable message.
Contact validation	Delivery and follow-up fail if customer data is wrong.	Validate name and Indian mobile number before allowing the order to move forward.
Capacity control	The kitchen has a maximum order size and should not accept unrealistic orders.	Enforce quantity range from 1 to 10 pizzas and explain the limit when exceeded.
Selection validation	Wrong item numbers create kitchen errors and billing disputes.	Validate base, pizza, and topping selections against the active menu catalogue.
Pricing control	Discount and GST mistakes directly affect revenue and customer trust.	Calculate subtotal, quantity discount, GST, and final payable from system prices only.
Payment confirmation	Staff need to know how payment will be settled.	Allow only Cash, Card, and UPI, and confirm the selected mode before saving the order.
Order persistence	Management needs clean data for revenue, demand, and menu analysis.	Save only completed orders in a consistent parseable format.

Acceptance Criteria

- The system completes a valid order without manual calculation.
- Every required edge case produces a controlled user-facing message.
- Menu items are loaded from the active source files at runtime and the system still works when menu content changes while preserving the expected format.
- GST is calculated after discount.
- The discount line appears only when quantity is greater than or equal to 5.
- The order log contains every required field and can be parsed later.
- No completed order is logged unless the full order and payment flow succeeds.

Drawbacks and Limitations

The current system scope is an MVP, not a production ordering platform. Its main limitations are:

- Text-file menus are fragile compared with database-backed menu management.
- `orders\_log.txt` is not safe for concurrent writes from many sessions.
- Local file storage makes analytics, backup, and recovery weak.
- There is no real online payment capture, only payment mode confirmation.
- There is no inventory control, prep-time estimate, kitchen display system, or rider dispatch workflow.
- There is no admin authentication, role-based access, or audit trail.
- There is no customer account, repeat-order history, loyalty, refund, cancellation, or complaint workflow.
- There is no address capture in the MVP scope, which is a major gap for a delivery-first outlet.
- There is no SLA monitoring for the 30-minute delivery promise.
- GST invoice compliance is simplified; a real business would need invoice numbering, tax records, and possibly ITC reconciliation.

At 1,000 orders, the MVP design breaks down operationally and technically. A flat text log becomes hard to search, summarize, lock safely, and recover from corruption. Multiple concurrent users can overwrite or interleave writes. The kitchen also needs queue prioritization, rider assignment, prep status, cancellation handling, and real-time stock visibility. For the production release, orders, menu items, customers, and order line items should move to a relational database, with admin analytics and controlled access.

Cost vs Value Analysis

Estimated Build Effort

Workstream	Estimated Hours	Notes
Requirements and economics analysis	10-14	Requirements, flow, financial modeling, scenario analysis
Menu parser and validation	3-5	Runtime file loading, malformed row handling
Step-based order flow	8-12	State management, screens, retries
Pricing, discount, GST, bill rendering	4-6	Formula implementation and bill UI
Payment mode flow	1-2	Three modes plus validation
Order logging	2-3	Parseable append format
Edge-case testing	4-6	All required invalid inputs plus swapped files
Total MVP build effort	22-34	Excludes production full-stack rebuild

Business Value

Value Area	Measurable Impact
Faster order taking	Reduces manual phone order time and repeated price explanation.
Pricing accuracy	Eliminates manual GST and discount mistakes.
Customer trust	Itemized bill makes final payable transparent.
Data capture	Creates about 1,380 monthly order records at the 60% capacity plan.
Menu decisions	Owner can identify top pizzas, toppings, average quantity, and discount usage.
Staffing decisions	Timestamped orders reveal peak hours for kitchen and rider planning.
Future AI readiness	Structured order history supports recommendation, forecasting, and admin analytics in the production release.

The MVP is worth building because even a simple local system creates immediate operational discipline. The highest-value output is not just the order screen; it is the clean order data that supports better pricing, staffing, and menu decisions.

Business Unit Economics

Assumption Set

Assumption	Value	Rationale
Outlet	Single SliceMatic outlet	Current operating model
Operating days/month	30	Reference model
Weekday orders/day at 60% capacity	38	Reference model
Weekend orders/day at 60% capacity	68	Reference model
Monthly orders at 60% capacity	1,380	22 weekdays and 8 weekend/holiday days
Menu prices	Ex-GST	GST is added at billing
GST rate	18%	Billing requirement used in this model
Discount	10% for quantity >= 5	SliceMatic promotional policy
Average base price	Rs.177.00	From provided menu file
Average pizza price	Rs.341.50	From provided menu file
Average topping price	Rs.50.00	From provided menu file
Average single-pizza basket	Rs.568.50	Base plus pizza plus topping
AOV used for model	Rs.850.34	Weighted from weekday/weekend demand plan
Estimated pizzas/order	1.50	Rs.850.34 / Rs.568.50

Monthly Fixed Costs

Cost Item	Monthly Cost
Kitchen rent	Rs.55,000
Equipment EMI	Rs.14,500
Electricity	Rs.12,600
Gas / LPG	Rs.5,550
Head chef	Rs.28,000
Kitchen helper	Rs.16,500
Counter and billing staff	Rs.18,000
Delivery riders - fixed component	Rs.32,000
Internet, POS, SaaS	Rs.2,800
Marketing - fixed	Rs.8,000
Packaging - fixed component	Rs.4,200
Miscellaneous / contingency	Rs.5,760
Total Fixed Costs	Rs.2,02,910

COGS and Gross Margin by Pizza Type

The reference COGS table is useful for ingredient logic, but its listed selling prices are lower than the active menu catalogue. The refined model below uses current menu pricing and the reference ingredient-cost structure.

Pizza Basket Example	Ex-GST Selling Price	Estimated COGS	Gross Margin	Gross Margin %
Thin Crust + Margherita + avg topping	Rs.498	Rs.136	Rs.362	72.7%
Thick Crust + Farm House + avg topping	Rs.548	Rs.163	Rs.385	70.3%
Cheese Burst + BBQ Chicken + Extra Cheese	Rs.677	Rs.219	Rs.458	67.7%
Average menu basket	Rs.568.50	Rs.170	Rs.398.50	70.1%

Category-level economics:

Category	Average Price	Estimated Food COGS	Gross Margin	Gross Margin %
Base	Rs.177.00	Rs.48.60	Rs.128.40	72.5%
Pizza selection	Rs.341.50	Rs.73.00	Rs.268.50	78.6%
Topping	Rs.50.00	Rs.10.00	Rs.40.00	80.0%
Packaging per pizza	Rs.0 revenue	Rs.18.00 cost	Negative	Not a revenue item
Delivery incentive per order	Rs.0 revenue	Rs.22.00 cost	Negative	Not a revenue item

Revenue Model at 60% Capacity

Metric	Weekday	Weekend/Holiday	Monthly Total
Days	22	8	30
Orders/day	38	68	1,380 orders/month
AOV	Rs.792	Rs.940	Rs.850.34 weighted average
Revenue/day	Rs.30,096	Rs.63,920	-
Monthly ex-GST revenue	Rs.6,62,112	Rs.5,11,360	Rs.11,73,472

GST treatment:

GST Item	Amount
Ex-GST revenue	Rs.11,73,472
Output GST at 18% charged to customers	Rs.2,11,225
Customer GMV including GST	Rs.13,84,697
Estimated input tax credit	Rs.20,000
Estimated net GST payable	Rs.1,91,225

GST is not revenue. It is collected from customers and remitted to the government after eligible input tax credit.

Contribution Margin Analysis

The refined model scales variable costs with the estimated number of pizzas per order.

Line Item	Per Order
Revenue, ex-GST	Rs.850.34
Ingredient COGS: 1.50 pizzas x Rs.131.60	Rs.196.84
Packaging: 1.50 pizzas x Rs.18	Rs.26.92
Delivery variable incentive	Rs.22.00
Blended payment/settlement fee: 1.8% of order value	Rs.15.31
Total variable cost/order	Rs.261.07
Contribution margin/order	Rs.589.27
Contribution margin %	69.3%

Monthly operating view at 60% capacity:

Metric	Amount
Monthly ex-GST revenue	Rs.11,73,472
Monthly variable costs	Rs.3,60,280
Monthly contribution	Rs.8,13,192
Monthly fixed costs	Rs.2,02,910
Estimated EBITDA	Rs.6,10,282
EBITDA margin	52.0%

Break-Even Analysis

$\text{Break-even orders/month} = \text{Fixed costs} / \text{Contribution margin per order}$
 $\text{Break-even orders/month} = \text{Rs.2,02,910} / \text{Rs.589.27} = 344.34$
 $\text{Break-even orders/day} = 344.34 / 30 = 11.48$

Rounded operating break-even: 12 orders/day.

At the 60% capacity plan of about 46 orders/day, SliceMatic operates at roughly 4.0x break-even. The model is therefore resilient as long as demand reaches even a modest fraction of planned launch volume.

Delivery Radius Economics

Radius	Economics	Recommendation
0-2 km	Fastest deliveries, low rider time, high SLA control	Serve from day one
2-4 km	Best trade-off between addressable demand and delivery time	Primary launch radius
4-6 km	More demand but weaker SLA and rider productivity	Add only after demand is stable
Above 6 km	Higher delivery time, cold-food risk, and rider cost	Avoid at launch

Recommended launch radius: 4 km. Expand to 5 km only when average order volume is at least 55 orders/day or when a third rider is added. The third rider adds about Rs.16,000/month fixed cost but protects delivery SLA and unlocks additional radius.

Scenario and Sensitivity Analysis

1. Break-even if rent rises to Rs.70,000/month

Current fixed cost: Rs.2,02,910.

Rent increase: Rs.15,000.

New fixed cost: Rs.2,17,910.

New break-even orders/month = $\text{Rs.2,17,910} / \text{Rs.589.27} = 369.8$
New break-even orders/day = 12.3

The model remains viable at Rs.70,000 rent.

Mathematically, if "unviable" means unable to break even at the 60% capacity plan of 1,380 orders/month, rent could rise to roughly Rs.6.65L/month before EBITDA reaches zero. Commercially, that is not a sensible rent threshold. For a small QSR, rent above about 10% of ex-GST revenue is a warning signal. At this revenue plan, rent above roughly Rs.1.15L-Rs.1.25L/month should be treated as strategically risky even though the spreadsheet still shows profit.

2. Impact of 40% aggregator orders at 25% commission

Assumption: Aggregator commission is charged on ex-GST order value.

Weighted commission/order = $40\% \times 25\% \times \text{Rs.850.34} = \text{Rs.85.03}$
New contribution margin/order = $\text{Rs.589.27} - \text{Rs.85.03} = \text{Rs.504.24}$
New break-even orders/month = $\text{Rs.2,02,910} / \text{Rs.504.24} = 402.4$
New break-even orders/day = 13.4

Aggregator channels are useful for demand discovery but materially reduce margin. SliceMatic should avoid becoming aggregator-dependent unless the order volume is incremental, not cannibalized from direct orders.

3. When is a third rider financially justified?

A third rider adds about Rs.16,000/month fixed cost.

Orders needed to cover third rider = $\text{Rs.16,000} / \text{Rs.589.27} = 27.2$ orders/month
Daily equivalent = 0.9 extra orders/day

Purely financially, one incremental order/day covers the rider. Operationally, the right trigger is stricter: hire the third rider when daily orders consistently exceed 55, when 4-6 km deliveries are accepted, or when the 30-minute SLA starts failing during peaks. At 55 orders/day, the extra nine orders/day versus the 46-order plan create about Rs.1.59L/month in contribution, far above the rider cost.

4. Impact of 10% discount on a 5-pizza order

Using a representative high-value order:

Metric	Without Discount	With 10% Discount
Ex-GST revenue	Rs.3,385.00	Rs.3,046.50
Discount	Rs.0	Rs.338.50
Estimated variable cost	Rs.1,067.93	Rs.1,061.84
Contribution margin	Rs.2,317.07	Rs.1,984.66
Contribution margin %	68.5%	65.1%

The discount reduces contribution by about Rs.332.41 after the small payment-fee saving. It is still profitable because delivery cost is spread over five pizzas, but it becomes a margin leak if customers would have ordered five pizzas anyway. It is a value-driver only if it increases order size, converts group orders, or fills underutilized kitchen capacity.

5. COGS inflation of 12%

Assumption: ingredient cost, not delivery incentive or payment gateway fee, rises by 12%.

Ingredient cost/order = Rs.196.84
Inflation impact/order = Rs.23.62
Monthly EBITDA loss at 1,380 orders = Rs.32,597
New contribution/order = Rs.565.65
Additional orders needed/month = 57.6
Additional orders needed/day = 1.9

SliceMatic needs about 2 extra orders/day to maintain the same EBITDA after a 12% ingredient inflation shock, assuming prices are unchanged.

6. BI metrics that improve profitability

Three high-impact metrics from order data:

- Contribution margin by base, pizza, topping, and combo. This identifies high-margin upsell combinations and low-margin menu items.
- Demand heatmap by hour, weekday/weekend, and payment mode. This improves staff scheduling, prep planning, and rider allocation.
- Discount usage versus quantity and AOV. This shows whether the quantity discount is increasing basket size or simply giving away margin.

Additional useful metrics include repeat customer rate by phone number, top-selling pizzas, topping attachment rate, average pizzas/order, payment-mode mix, and delivery-zone profitability once addresses are captured in the production release.

Recommendations for MVP and Production Release

- Keep the MVP pricing constants in one place: GST rate, discount rate, max quantity, and payment modes.
- Build the menu loader defensively before building UI polish.
- Log every monetary field explicitly so analytics does not need to recompute historic orders after business rules change.
- In the production release, model database tables as `menu\_categories`, `menu\_items`, `orders`, `order\_line\_items`, and optionally `customers`.
- Add address and delivery radius validation in the production release, because the MVP scope omits a delivery-critical field.
- Prefer an AI recommendation feature if the team wants the strongest business link: it directly uses order history to increase AOV and repeat purchase quality.

Final Position

SliceMatic is economically attractive at the 60% capacity target, even under a more conservative contribution-margin model. The product should be built first as a robust, validated ordering workflow rather than as a visually decorative app. The major business risk is not whether the MVP can calculate a bill; it is whether the system captures clean enough data to support operational decisions, menu optimization, delivery radius control, and future customer personalization.